

Name & Parameter

↳ method signature

Same name with different structures → Overload
~~Signature~~

Return type is not a part of
Signature

Different Return type alone

Cannot make method overload

{ methods with same name, param
different return type in same
class is illegal } → not
allowed

Overriding

return type → compatible

It may have return type

Subclass type of original

↳ Covariant type

private, final, static or abstract

Compiler knows
which method → static binding
should call
Compilation time

or else

compiler calls.
based on actual
type of object → Dynamic binding
Run time

Program run

Dynamic binding
→ method

(Time consuming)

VM → Version of method
according to actual
object

Check in subclasses
or else subclassing

Virtual method →
polymorphism

method table

method signature
and its class

Actual method
for each class

Virtual machine
initial method
table

{ Given class
is loaded }

Method is added

VM now
lookup of table

One twist

superclass

method

Virtual machine consults
method table of superclass

Override a method

Subclass method should
appear as visible as
superclass method

Preventing Inheritance: Final class & methods

Final class

If you need to make sure no one will use
your class as their subclass.

→ final

final class sample

{

}

final class cannot be ~~not~~ inherited.

All the methods inside final class is final
method

But what if you need to make only
some method final

↓
Put final on that method also

No one should
mess up class or
method → final class

Some programmers → final keyword
avoid overhead of dynamic binding

If a method is not overridden
compiler optimize method calling
→ inlining

e.getName() → e.name
change

CPU hate branching

Interferes with strategy of
prefetching instructions while
processor waiting one

CPU strategy

Fetch

Process

Branching

second
one

strategy

Fetch

→ interrupts it

~~Example~~ em

Inlining does not occur in compilers

- * It compiler finds our method ^{non final} can be potential of overridden Because compiler cannot find override one

Virtual machine can do pretty good job

Just in time compiler in virtual machine can take method often called simple not overridden for inlining

Here's twist Virtual machine in its execution finds that method

Is overriden insulting

Optimizes under the inlining

Enumerations } -> final
Records

Casting

Casting is forcing someone to target type

Primitive References

float -> int

float x = 12.92;

int y = (int) x;

Discarding 0.92 in float

Object References

Done only when you
actually forget temporarily object
type

class Employee

{
protected int id;
protected double salary;

}
class Manager extends Employee

{
protected double bonus;

}

Employee[] employees = new Employee[5];

employees[0] = new Manager();

employees[1] = new Employee();

employees[2] = new Employee();

when you try to access employees[0].bonus X
ERROR

It forgets the object type

Manager boss = (Manager) employees[0];

↓
Able to access bonus

I need correct type
==

Java -> strongly typed language

Type

Every variable has its type

type defines what it can do

If you give subclass reference to
superclass variable

~~class Employee~~

Employee -> Superclass variable

Employee e = new Manager();

promising less

Manager boss = (Manager) ^{Employee} simple[0];

error

Manager boss = (Manager) simple[0];

promise more

we need type cast

If you broke promise



class cast exception

Good practice down

is steps (eg instance of marsh)

1

3

Compiler will not let you
succeed → no chance
to cut

You can cut with inheritance
Hierarchy

instance of chess

superclass → subclass
(down country)

Convert type of object
is not necessary

Convert type of object is not
necessary unless access method
unique to Subclass

Pattern match for instance of

Prac 1

Jan 16

if (stuff is instance of man boss)
 { boss.set bonus }

}

if stuff is → instance of
 man

boss.set he set

You ship call

if (stuff is not instance
 of man)

if statement skipped

→ skip

main horse

if (base inst of engine)

ERROR

if (base inst of engine) ✓

Backward compatibility

part 2

instance of part

if (m inst of main m ss m.getBase() score)

run
Executes only left side

H

↓

compile time error

conditionalize

inst of
main m

m.setBase() * 0
✓

X

100 variable defined by init of
pattern can share
a file

Protected Access

Private → Access within class
not even subclasses
can access it

Public → Access Everywhere

Package → Access with same
package
(default)

Times to access by researching

Protects

↓
interfaced class + classes
in package

protected - strict $\Rightarrow$ finally
caution

may broke encapsulation

protected method

- * class may declare a method protected
- * subclass can be built to use method
- other class cannot.

protects

↓

all subclasses

to class with
package
